Research Agent

Abstract

LLM 기반 에이전트의 성능은 최근 빠르게 고도화되고 있으며, 특히 코딩과 수학처럼 정답 구조가 비교적 명확한 닫힌 환경에서는 이미 인간 전문가에 근접한 문제 해결 능력을 보이고 있다. 특히, GPT-5.4 계열 모델은 reasoning, coding, 장문맥 처리, 도구 사용, 그리고 다단계 agentic workflow 수행 능력에서 뚜렷한 개선을 보였으며, 이는 문제 파악, 해결 전략 수립, 실험 설계로 이어지는 연구 과정의 상당 부분을 모델 중심으로 조직할 수 있음을 시사한다. 예를 들어, Karpathy의 autoresearch는 코드 수정, 단기 학습, 검증, 채택/폐기를 반복하는 자율 실험 루프를 보여주었고, 수학 영역에서도 Erdős Problems 데이터베이스의 일부 open question에 대해 semi-autonomous 해결 사례가 보고되었다. 이러한 흐름을 바탕으로, 문제 파악과 해결 전략 제안은 강력한 reasoning 모델이 담당하고, 실제 실험 수행과 결과 검증은 시스템 에이전트(codex 등)가 담당하는 자율 연구 에이전트용 harness를 제안한다. 제안하는 harness는 문제 정의, 가설 생성, 실험 계획, 코드 수정, 실행, 평가, 결과 기록의 전 과정을 반복 가능한 루프로 구성하며, 강력하고 범용적인 reasoning 모델은 계획과 의사결정을 담당하고 시스템 에이전트는 실행 안정성, 재현성, 로그 축적, 실패 복구를 담당하도록 설계된다. 이를 통해 본 프레임워크는 상위 reasoning 모델의 추론 능력을 실제 실험 자동화와 연결하면서도, 연구 과정 전반의 신뢰성과 통제 가능성을 함께 확보하는 것을 목표로 한다.

1 Workflow

1.1 Overview

본 프레임워크는 토론(debate)과 실행(execution)을 번갈아 수행하는 순환형 2단계 제어 루프로 구성된다. 각 사이클 c에서 시스템은 원래 문제 설명, 이전 사이클에서 누적된 요약, 그리고 최신 실험 결과를 바탕으로 사이클별 문제 문맥 P_c를 구성한다. 최상위 제어기는 먼저 debate section을 호출하여 후보 연구 방향을 생성하고 검토한 뒤, experiment section을 호출하여 선택된 방향을 실행 가능한 코드, 단기 ablation, 장기 main run, 그리고 최종 요약으로 진행한다.

Algorithm 1는 사이클 수준의 전체 제어 절차를 보여준다. 각 사이클은 현재의 문제 문맥에서 시작하여 debate section과 experiment section을 순차적으로 수행한다. 실험 단계의 출력은 단순한 종료 보고서로 사용되지 않고, 다음 사이클의 초기 문맥을 구성하는 요약으로 압축된다. 이를 통해 프레임워크는 실패 사례, 부분적 성공, 구현 과정에서 축적된 지식을 다음 반복에 반영하는 제한된 형태의 iterative research를 수행할 수 있다.

Algorithm 1 Overall framework

Require: Bootstrap context P_c, auto-cycle count C, debate max rounds R, pre-main ablation rounds A, main-run recovery retries K

for $c = 1$ to C **do**
  run `debate_section`
  run `experiment_section`
end for
Transition to `WAITING_FOR_NEXT_PROBLEM`

1.2 Debate section

Debate section의 목적은 단순한 자유 형식의 토론을 수행하는 것이 아니라, 강력하고 보다 범용적인 모델(ChatGPT - pro extended thinking)을 주요 추론 도구로 사용하여 원시적인 문제 서술을 실제 실험으로 옮길 수 있는 계획으로 변환하는 데 있다. 이를 위해 프레임워크는 사이클별 debate log인 $\mathcal{D}_c$를 컨텍스트로 명시적으로 유지하며, 모든 제안, 비판, 수정, 의사결정 과정을 기록한다. 이 단계는 먼저 병렬 ideation으로 시작하여 초기 탐색의 다양성을 확보하고, 이후 narrowing 단계를 통해 여러 제안 중 실현 가능성이 높은 소수의 방향으로 압축한다. 그 다음 라운드에서는 criticism과 refinement를 반복하면서 실현 가능성, 식별 가능성, 계산 비용, 구현 범위, 평가 설계 측면의 약점을 본격적인 코드 수정 이전에 드러내도록 한다. Algorithm 2에서 전체 알고리즘을 확인할 수 있다.

여기서 `literature_surveyor` 누적된 debate trace를 바탕으로 현재 제안과 가장 밀접한 선행연구를 정리하여 debate log에 주입하는 역할을 한다. `critic` 모듈은 누적된 debate trace를 바탕으로 현재 제안의 결함, 누락된 가정, 취약한 연결고리를 지적하고, `refinement` 모듈은 그 비판에 대응하여 방향을 수정한다. 이후 final narrowing 단계는 갱신된 상태를 다시 압축하여 실행 가능한 수준의 실험 계획으로 정리한다. `action_decider`는 현재까지 축적된 근거가 실제 실험을 시작하기에 충분한지, 탐색 폭을 넓히기 위해 ideation을 다시 열어야 하는지, 혹은 추가 토론이 필요한지를 판단한다.

1.3 Experiment section

실행 단계에 진입하면 선택된 계획은 고수준 reasoning과 저수준 구현 사이를 연결하는 구조화된 중간 산출물로 변환된다. Experiment section은 pseudocode specification과 draft note를 먼저 작성하고, 이후 main experiment code를 이 중간 표현에 맞게 정렬한다. 이는 실험 절차의 의도를 점검 가능하게 만들고, 계획과 구현 사이의 드리프트를 줄이며, 시스템 에이전트가 이후 코드 수정에서 참조할 명시적인 목표를 제공한다. 즉, 느슨한 자연어 목표를 곧바로 실행 환경에 전달하는 대신, 고비용 실행 전에 실험 의도를 더 명확한 표현으로 고정하기 위함이다.

Algorithm 3은 이후의 staged execution 절차를 보여준다. 장시간 main run을 시작하기 전에 프레임워크는 A회의 짧은 ablation을 수행하여 불안정한 하이퍼파라미터를 조기에 식별하고, 명백한 구현 오류를 발견하며, 기본 설정값에 대한 경험적 근거를 확보한다. 이 단계의 결과는 tuning artifact로 정리되며,

이는 최종 main run의 설정을 제약하는 역할을 한다. 여기서 pre-main 단계의 목적은 성능을 끝까지 최적화하는 것이 아니라, 피할 수 있는 실패 모드를 줄이고 이후의 고비용 실험이 보다 안정적으로 수행되도록 만드는 데 있다.

long main run은 가장 비용이 큰 단계로 간주되므로, 명시적인 recovery 메커니즘과 함께 운용된다. 실행이 실패했을 때 시스템은 단순히 처음부터 재시작하지 않고, 먼저 raw failure artifact를 분석하고 recovery note를 작성한 뒤, 이 실패가 재시도 가능한 유형인지와 재실행이 정당화되는지를 판단한다.

반면 main run이 operator에 의해 중단된 경우, 프레임워크는 이를 자동 실패와 동일한 방식으로만 처리하지 않고, operator의 명시적 판단을 통해 후속 조치를 결정한다. 이를 위해 시스템은 중단 직후 CLI에서 후속 동작을 질의하고, 그 선택과 근거를 `interrupt_decision_##.md`에 기록한다. 첫째, `ANALYZE_NO_LEARNING`은 학습이 실제로 진행되고 있지 않거나, 이전 상태와 비교해 의미 있는 개선이 관찰되지 않는다고 판단될 때 선택된다. 이 경우 시스템은 해당 main run을 실패로 처리하고, operator가 관찰한 정체 원인과 근거를 함께 analysis 단계로 전달한 뒤 실행 루프를 종료한다. 즉 이 선택의 목적은 중단된 run을 성공적 artifact로 간주하는 것이 아니라, 더 이상의 실행이 유의미한 진전을 만들지 못한다는 판단 자체를 분석 입력으로 보존하는 데 있다. 둘째, `OPTIMIZE_AND_RETRY`는 중단의 원인이 학습 부재 자체라기보다 실행 속도, 자원 사용, 또는 relaunch readiness와 같은 운영상의 비효율에 있다고 판단될 때 사용된다. 시스템은 중단 시점의 raw artifact와 현재 코드를 함께 검토하여 `speed_optimization_##.md`를 작성하고, 실제로 적용 가능하며 재실행 준비가 완료된 최적화가 도출된 경우에만 이를 반영한 뒤 main run을 다시 시작한다. 반대로 실질적인 최적화 여지가 없거나 재실행 준비 상태에 도달하지 못하면, 시스템은 그 한계와 근거를 남기고 해당 run을 분석 대상으로 전환한 뒤 루프를 종료한다. 셋째, `RERUN_UNCHANGED`는 인터럽트가 우발적이거나 외생적인 이유로 발생하여 코드나 설정을 바꾸는 것이 오히려 비교 가능성을 해칠 때 선택된다. 이 경우 시스템은 어떠한 수정도 가하지 않고 동일한 코드와 설정으로 main run을 다시 수행한다.

실험의 마지막 단계에서 프레임워크는 문제 문맥 P_c, debate trace인 $\mathcal{D}_c$, tuning artifact, main-run output, 그리고 recovery log 전체를 종합적으로 분석하고, 그 결과를 experiment summary로 압축한다. 이 summary는 사람이 읽을 수 있는 기록이자 다음 사이클의 초기 문맥으로 재사용되는 기억 단위로 기능한다. 따라서 시스템은 매번 전체 상호작용 이력을 그대로 재생하지 않고도, 핵심 상태를 축적하면서 반복적인 연구를 수행할 수 있다.

1.4 Context Management

프레임워크가 장기적이고 다단계적인 연구 과정을 수행할 때, 어떤 정보를 언제 어떤 해상도로 각 모듈에 노출할지를 제어한다. 이는 컨텍스트 길이의 무제한 증가를 피하고, 이전 탐색에서 형성된 가설과 실패 원인을 다음 단계로 안정적으로 전달하기 위함이다. 여기서 핵심은 전체 상호작용 이력을 무차별적으로 재주입하는 것이 아니라, cycle c의 문제 cotext P_c와 debate trace $\mathcal{D}_c$를 구성하고, 이를 중심으로 현재 단계에 필요한 산출물만 선택적으로 context로 사용하는 데 있다. 이때 c는 현재 연구 cycle의 인덱스, P_c는 해당 cycle이 풀어야 할 문제를 기술한 문맥, $\mathcal{D}_c$는 같은 cycle에서 누적된 제안, 비판, 수정, 의사결정의 기록을 의미한다.

Problem Context P_c 새 cycle이 시작되면 orchestrator는 먼저 문제 context P_c 를 구성한다. 첫 cycle에서는 사용자가 현재 문제 상황을 자세하게 기술한 `problem` + `init_experiment.md` 이 직접 P_1 로 사용되고, 이후 cycle에서는 실험 로그, 초기 문제 상황, 이전 cycle의 debate log 를 사용하여 P_c 를 구성한다. 구체적으로, P_c 는

$$P_c = P_1 + \widehat{\mathcal{D}}_{c-p_{limit}} + (c-p_{limit})\texttt{_experiment.md} + \cdots + \widehat{\mathcal{D}}_{c-1} + (c-1)\texttt{_experiment.md}$$

여기서 p_{limit} 은 context 구성을 위해 참고할 최대 cycle window 길이이고, $\widehat{\mathcal{D}}$ 는 한 cycle의 debate log 를 요약한 cycle level decision memo, c`_experiment.md` 는 cycle c 의 최종 실험 요약 파일 $\star$`_experiment.md` 를 의미한다.

Debate Context $\mathcal{D}_c$ `debate_section` 에서의 context 관리를 위한 debate log $\mathcal{D}_c$ 기록한다. 토론 과정에서 고급 reasoning backbone 을 사용하고, 그 출력이 매우 길기 때문에 안정적인 정확도로 전체 시스템을 운용할 수 있는 context 길이를 초과할 우려가 있으므로 매 토론 step에서의 응답 결과물을 요약하여 $\mathcal{D}_c$ 에 계속 append하고, $\mathcal{D}_c$ 는 cycle c 에서 계속 토론을 위한 context로 사용한다.

Experiment Context 이렇게 해석된 P_c 와 최신 debate trace인 $\mathcal{D}_c$ 는 실험 설계 단계에서 다시 구조화된 산출물로 변환된다. 프레임워크는 short ablation에 들어가기 전에 먼저 `pseudocode.tex`와 `draft.tex`를 작성하게 하며, 이 과정에서 인간이 제공한 `init_pseudocode.tex`와 `init_draft.tex`를 각각 abstraction, completeness, style의 reference로 사용한다. 이는, 자유 형식의 자연어 토론 결과를 곧바로 코드 수정으로 넘기지 않고, self-contained한 중간 표현으로 먼저 고정함으로써 이후 구현과 실행이 참조할 명시적 목표를 만들기 위함이다.

Prepare phase에서의 각 ablation round는 P_1, P_c, $\mathcal{D}_c$, 직전 main experiment 로그 전체 ($c = 1$ 에서는 사람이 수행했던 직전 실험 로그), 그리고 같은 cycle 안에서 생성된 모든 prior ablation log, `pseudocode.tex`와 `draft.tex`를 읽는다. 특히 prior ablation log는 별도 요약 없이 모델이 직접 읽고 비교하도록 한다. 그 결과 각 round는 이전 tuning 판단을 덮어쓰는 대신, 이미 관찰된 실패 모드와 유망한 설정을 누적적으로 상속한다.

일련의 ablation cycle 이후에는 선택된 설정을 일회성 CLI 인자로 남겨두지 않고 training code의 default로 직접 승격시키도록 강하게 요구된다. 이는 tuning의 결과가 실행 시점에 별도 외부 컨텍스트 없이도 자동으로 재현되도록 하기 위함이다.

analysis phase에서는 latest problem context, latest debate log, latest experiment log, tuning log, long main run raw artifact, recovery log를 종합하여 최종 `experiment.md`를 작성한다. 이 문서는 사람이 읽을 수 있는 결과 요약이자 다음 cycle의 초기 problem context 구성을 위해 사용된다.

Algorithm 2 `debate_section`

Require: Bootstrap context P_c, auto-cycle count C, debate max rounds R, pre-main ablation rounds A, main-run recovery retries K

Resolve problem context P_c
Check external LLM readiness
Initialize debate log $\mathcal{D}_c$
Scribe initial problem context into $\mathcal{D}_c$
Run `idea_generator_1` and `idea_generator_2` in parallel
Scribe both idea outputs into $\mathcal{D}_c$
Run `literature_surveyor` on the accumulated debate log
Scribe literature survey output into $\mathcal{D}_c$
Run `narrowing` on the accumulated debate log
Scribe narrowing output into $\mathcal{D}_c$
for $r = 1$ to R **do**
  Run `critic` on the accumulated debate log
  Scribe critic output into $\mathcal{D}_c$
  Run `refinement` on the accumulated debate log
  Scribe refinement output into $\mathcal{D}_c$
  Run `final_narrowing` on the accumulated debate log
  Scribe final narrowing output into $\mathcal{D}_c$
  Run `action_decider` on the accumulated debate log
  if `action_decider` returns `START_EXPERIMENT` **then**
    break
  else if `action_decider` returns `REOPEN_IDEATION` **then**
    Run `idea_generator_1` and `idea_generator_2` in parallel
    Scribe both idea outputs into $\mathcal{D}_c$
    Run `literature_surveyor` on the accumulated debate log
    Scribe literature survey output into $\mathcal{D}_c$
    Run `narrowing` on the accumulated debate log
    Scribe narrowing output into $\mathcal{D}_c$
  else if `action_decider` returns `CONTINUE_DEBATE` **then**
    pass
  end if
  if $r = R$ and no `START_EXPERIMENT` **then**
    Terminate
  end if
end for

Algorithm 3: `experiment_section`

Require: Bootstrap context P_c, auto-cycle count C, debate max rounds R, pre-main ablation rounds A, main-run recovery retries K

Enter experiment workflow using P_c and $\mathcal{D}_c$
Write `pseudocode.tex`, `draft.tex` from P_c and $\mathcal{D}_c$
Modify main experiment code as `pseudocode.tex`
for $a = 1$ to A **do**
  Run codex preparation / short ablation round a
  Read all prior ablation logs from the same cycle
  if $a = A$ **then**
    Write `tuning.md`
    Apply selected defaults directly in training code
  end if
end for
Initialize automatic failure-recovery counter $t \leftarrow 0$
while true do
  Run the bare long main experiment
  if main run succeeds **then**
    break
  end if
  if main run is interrupted by a human **then**
    Query the operator on CLI:
      `ANALYZE_NO_LEARNING`, `OPTIMIZE_AND_RETRY`, or `RERUN_UNCHANGED`
    Write `interrupt_decision_##.md`
    if operator chooses `ANALYZE_NO_LEARNING` **then**
      Mark the run for analysis with the operator' s judgment and evidence
      break
    else if operator chooses `OPTIMIZE_AND_RETRY` **then**
      Run codex speed review on the interrupted raw artifact and current code
      Write `speed_optimization_##.md`
      if speed optimization is feasible and relaunch-ready **then**
        Apply the selected speed optimizations
        continue
      else
        Mark the run for analysis with the speed-limit judgment and supporting evidence
        break
      end if
    else if operator chooses `RERUN_UNCHANGED` **then**
      continue
    end if
  end if

if $t = K$ **then**
  break
end if
$t \leftarrow t + 1$
Run codex recovery on the failed raw artifact
Write `recovery_##.md`
Read all prior recovery logs from the same cycle
if recovery is not retryable or not relaunch-ready **then**
  break
end if
end while
Run codex analysis on problem context, debate log, tuning log, main-run artifact, interrupt decision logs, speed-optimization logs, and recovery logs
Write final experiment summary `experiment.md`
if the cycle ends in failure **then**
  stop
end if
Save latest experiment summary as the next-cycle context seed
Transition to `WAITING_FOR_NEXT_PROBLEM`

2 LLM Knowledge Base